

Compiler Design - Module 1

Advanced Level Study Guide

Bootstrapping • Compiler Construction Tools • Detailed DFA/NFA Conversion

1. Bootstrapping

Definition

From the PDF:

Bootstrapping is used to produce a self-hosting compiler. A self-hosting compiler can compile its own source code. A compiler is characterized by Source language, Target language and Implementation language.

Bootstrapping means:

- Writing a compiler using the same language that the compiler will compile.

Languages in Bootstrapping

Term	Meaning
Source language	language to compile
Target language	machine code
Implementation language	language used to write compiler

Bootstrapping Process

From notes:

1. Create compiler for subset S of language L using machine A
2. Write compiler for L using L
3. Compile it to get final compiler

Bootstrapping Diagram - T-diagram

PDF shows T-diagram representation.

Meaning of T-diagram:

Source

|

Compiler

|

Target

Example:

```
L
-----
C
-----
A
```

Means: Compiler for language L produces code for machine A

Bootstrapping Example

Goal: compiler for language L on machine A

Step 1:

```
S compiler written in A
```

Step 2:

```
L compiler written in S
```

Step 3:

```
Compile L using S → get full L compiler
```

Diagram (from PDF):

```
L(S,A) + S(A,A) → L(A,A)
```

Meaning: Compiler written in S compiled using S gives final compiler.

Cross Bootstrapping

PDF also shows compiler for another machine.

Steps:

1. Convert compiler to target machine B
2. Use cross compiler
3. Generate compiler for B

Diagram:

```
Machine A → cross compiler → Machine B
```

Used when:

- new processor
- new OS
- new language

2. Compiler Construction Tools

Overview

From PDF:

Compiler writers use software tools to implement phases of compiler such as parser generator, scanner generator, code generator etc. These tools reduce complexity.

Types of Compiler Construction Tools

1. Scanner Generator

From PDF:

Input → regular expressions

Output → lexical analyzer

Diagram:

Regular Expression

↓

Scanner Generator

↓

Lexical Analyzer

Example:

digit → [0-9]

id → letter(letter|digit)*

Used to build token recognizer.

2. Parser Generator

From PDF:

Input → grammar

Output → syntax analyzer

Diagram:

Grammar

↓

Parser Generator

↓

Parser

Example grammar:

$E \rightarrow E + T$

$T \rightarrow T * F$

$F \rightarrow id$

Output → parser.

3. Syntax-Directed Translation Engine

From PDF:

Input → parse tree

Output → intermediate code

Diagram:

```
Parse Tree
↓
Translation Engine
↓
Intermediate Code
```

Example:

```
a = b + c
```

Intermediate code:

```
t1 = b + c
a = t1
```

4. Automatic Code Generator

From PDF:

Input → intermediate language

Output → machine code

Diagram:

```
Intermediate Code
↓
Code Generator
↓
Machine Code
```

Example:

```
t1 = b + c
```

Machine code:

```
ADD R1, R2
STORE a
```

5. Data Flow Analysis Engine

From PDF:

Used in optimization

Gathers information about values in program

Diagram:

Program



Data Flow Engine



Optimized Code

Example:

a = 5

a = 5

Optimized:

a = 5

6. Compiler Construction Toolkit

From PDF:

Collection of tools used to build compiler

Diagram:

Toolkit

■ Scanner

■ Parser

■ Translator

■ Optimizer

■ Code generator

3. Detailed DFA / NFA Conversion

Overview

From notes:

Regular Expression $\rightarrow$ NFA $\rightarrow$ DFA $\rightarrow$ Lexical Analyzer

Used in lexical analysis.

Step 1: Regular Expression $\rightarrow$ NFA

Example regex:

`a(b|c)`

NFA:

`q0 --a--> q1`

`q1 --b--> q2`

`q1 --c--> q2`

NFA properties:

multiple transitions

epsilon allowed

Step 2: Epsilon-Closure

Find reachable states.

Example:

$\epsilon(q_0) = \{q_0, q_1\}$

Used in conversion.

Step 3: NFA $\rightarrow$ DFA (Subset Construction)

From theory used in lexical analyzer.

Steps:

1. start state = ϵ -closure
2. create new state sets
3. repeat until done

Example NFA:

`q0 --a--> q1`

`q0 --a--> q2`

`q1 --b--> q3`

`q2 --b--> q3`

State sets:

D0 = {q0}

D1 = {q1, q2}

D2 = {q3}

DFA:

D0 --a--> D1

D1 --b--> D2

Diagram:

a b

D0 -----> D1 -----> D2

Step 4: DFA Minimization

Remove duplicate states.

Example:

q1 and q2 same → merge

Result → smaller DFA.

Complete Conversion Diagram

Regular Expression

↓

NFA

↓

Subset Method

↓

DFA

↓

Minimization

↓

Lexical Analyzer

This flow is used in compiler lexical analyzer.

Summary: Advanced Module 1 Topics

Key Concepts Covered

Topic	Key Concepts	Application
Bootstrapping	Self-hosting compiler, T-diagram, cross-compilation	Compiler portability
Compiler Tools	Scanner/Parser generators, code generators	Automated compiler construction
DFA/NFA Conversion	Regex to NFA, subset construction, minimization	Lexical analyzer design

Learning Path Progression

- Basic Level:** Understand fundamental concepts like tokens, phases, and simple patterns
- Intermediate Level:** Learn detailed examples, algorithms, and practical applications
- Advanced Level:** Study theoretical foundations like bootstrapping, automation tools, and formal methods

This is an advanced-level study guide for Module 1 - Compiler Design. Topics covered include bootstrapping techniques, compiler construction automation tools, and detailed formal conversion procedures from regular expressions to lexical analyzers.